

COMPETITIVE INTELLIGENCE

code-search 竞品与定位分析报告

基于当前仓库文档，系统比较 GitHub Code Search、Sourcegraph、CodeGraphContext、Glean、GitNexus、CodeGraph，并用 Agent Jobs-to-be-Done、战略画布和 ERRC 框架判断本项目的竞争优势。

产品假设

给 Agent 用的
local-first、git-first 代码搜索与跳转工具

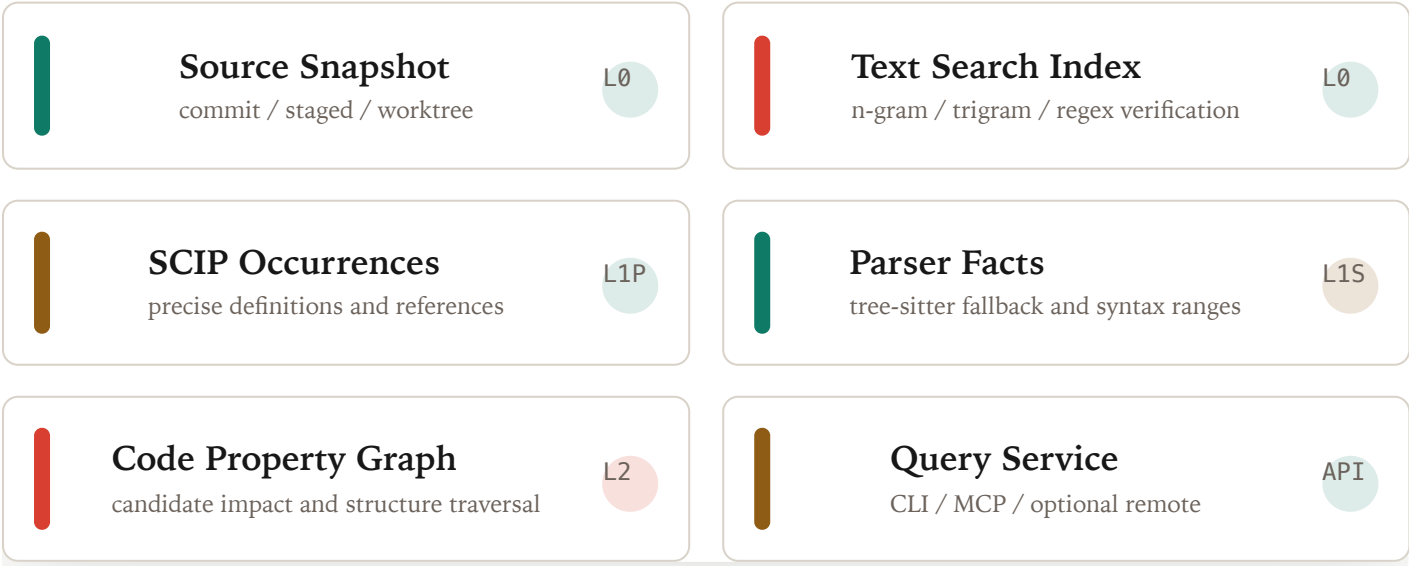
核心差异

每个结果都必须带
snapshot、range、producer、reliability

报告日期

2026-05-28

TARGET ARCHITECTURE



01

结论摘要

`code-search` 最有价值的位置不是“再做一个代码图”或“再做一个 grep 包装”，而是成为 Agent 的可验证代码证据层：快速给出搜索、跳转、读取、引用、影响候选，并明确告诉 Agent 哪些是事实，哪些只是候选。

战略判断

项目应继续坚持 deterministic-first。embedding、hybrid search、自动上下文组装可以成为 opt-in 扩展，但不能进入默认定位，否则会削弱“可信证据层”的差异化。

1

竞品最强处

GitHub/Sourcegraph 强在大规模搜索和 precise navigation；
GitNexus/CodeGraph 强在 Agent graph 与 MCP 工作流。

2

市场空位

现有工具普遍没有把 dirty/staged/commit、本地验证、LLM reliability instruction 统一成 CLI 契约。

3

产品切入

先做 L0 搜索/读取/路径与统一 JSON，再做 freshness/hook/watcher，最后接 SCIP 和 graph candidate。

02

项目定位

一句话定位

`code-search` 是面向人类开发者和 LLM Agent 的本地优先代码搜索与跳转系统，让 Agent 像 IDE 一样完成搜索、跳转、读取、引用和影响分析。

- Local first
- Git first
- Evidence first
- Deterministic first
- Remote optional

不是哪些产品

不是默认 embedding/semantic similarity 引擎。

不是把 tree-sitter 推断调用图包装成精确事实的 graph 产品。

不是只能看默认分支的远程 Web search。

不是自动生成任务上下文包的黑箱 runtime。

可靠性契约

级别	来源	适用命令	Agent 应如何使用
L0 source_fact	源码文本、路径、git/filesystem	find、grep、files、read、changed	可作为可验证源码证据，但仍需保留 snapshot、file hash、range。
L1P precise_fact	SCIP、语言服务、编译器索引	defs、refs、symbols	可作为 IDE 级跳转结果，前提是 producer 和 freshness 校验通过。
L1S parser_fact	tree-sitter AST	symbols、fallback defs	作为语法事实，不等同于 semantic reference resolution。
L2 inferred_candidate	AST heuristic、search-based inference、framework bridge	calls、callers、impact traversal	只能缩小搜索范围，必须用 read 验证后再推理。

竞品全景

GitHub Code Search / Blackbird

定位：GitHub 平台上的大规模代码搜索。

- Rust engine
- n-gram
- regex
- web scale

功能：代码文本搜索、符号式查询体验、跨仓库搜索、Web UI。

用法：开发者在 GitHub UI/API 中搜索托管代码，适合远程默认分支和平台内 corpus。

算法：code-specific n-gram/regex 检索，强调标点符号、无 stemming、无 stop words。

架构：大规模 Rust 搜索引擎与持续更新索引系统。

对本项目启发：文本搜索必须是专用层，regex 需要候选预筛和源码 range verification。

Sourcegraph / Zoekt / SCIP

定位：企业级代码搜索和 precise code navigation。

- Zoekt
- trigram
- SCIP
- code intel

功能：代码搜索、跳转定义、引用、批量代码理解和企业级 Web/API。

用法：团队把仓库接入 Sourcegraph，通过 Web/API 做搜索和导航。

算法：Zoekt trigram text index；precise navigation 依赖 SCIP indexer；缺 precise index 时使用 search-based fallback。

架构：搜索层和代码智能层分离，索引器由语言生态或构建流程产出。

对本项目启发：find/grep 和 defs/refs 必须分层，fallback 必须标注。

CodeGraphContext

定位：把代码转换为可查询 property graph。

tree-sitter

SCIP ingestion

KuzuDB

MCP

功能：graph builder/linker、CLI、MCP server、watcher、便携 bundle。

用法：先 ingestion，再通过 CLI/MCP 或图数据库后端查询结构关系。

算法：tree-sitter 与 SCIP ingestion 生成 nodes/edges，再进入 property graph。

架构：KuzuDB、FalkorDB、Neo4j 等可插拔 graph backend。

对本项目启发：graph 是结构遍历层，不应该替代 text index 或 precise occurrence index。

Glean

定位：typed, schema-defined code facts。

typed facts

schema

query

IDE/tools

功能：把 indexer 产出的代码事实建模为类型化 schema，供 IDE 和工具查询。

用法：构建流程或 indexer 产出 facts，查询层按 schema 消费。

算法：重点不是全文搜索，而是事实抽取、schema 约束和查询。

架构：事实数据库与查询系统，强调可演进 schema。

对本项目启发：graph schema 不能是随意 JSON，所有边都要带 producer、版本和 source snapshot。

GitNexus

定位：本地/浏览器 + MCP 的 Agent code graph workflow。

DAG ingestion

LadybugDB

hybrid search

skills/hooks

功能：analyze pipeline、graph、MCP/HTTP/CLI 查询、impact、rename、tool map、route map。

用法：本地分析仓库后，通过 MCP、HTTP bridge 或 Web UI 面向 Agent 使用。

算法：多阶段 ingestion DAG，包含 routes、tools、ORM、crossFile、MRO、communities、processes；部分能力结合 BM25/vector hybrid search。

架构：内存 KnowledgeGraph 加载到 LadybugDB，全局 registry 记录 repo。

对本项目启发：DAG 化 ingestion 和 Agent ops 值得学习；embedding/hybrid search 应保持 opt-in。

CodeGraph

定位：pre-indexed knowledge graph for agents。

SQLite

FTS5

watcher

provenance

功能：search、context、callers、callees、impact、node、status、files 等 MCP 工具。

用法：安装后在本地预索引，让 Claude Code、Codex、Cursor、OpenCode 等 Agent 查询。

算法：SQLite FTS5 全文搜索，tree-sitter 解析 20+ 语言，framework-aware routes 和 bridge edges。

架构：本地 SQLite 数据库、OS watcher debounce 增量同步、MCP/instructions 集成。

对本项目启发：local-first graph、watcher 和 provenance 值得学习，但不能把 graph 结果表述为 authoritative source。

功能与用法分析

目标命令面

```
code-search find "createCodeContextEngine"  
code-search grep "handle[A-Z].*"   
code-search files "src/**/*.rs"  
code-search find-path "runtime"  
code-search tree src --depth 3  
code-search read src/main.rs:10-40  
code-search defs authenticate_user  
code-search refs createCodeContextEngine  
code-search calls authenticate_user  
code-search index status  
code-search watch --status
```

和传统工具的差异

相对 **grep/rg**：不仅返回文本匹配，还返回 snapshot、range、producer、freshness。

相对 **IDE**：以 CLI/JSON/MCP 给 Agent 使用，不需要人类 UI 才能拿到跳转信息。

相对 **graph** 产品：graph 只是候选和结构层，不能替代源码事实和 precise occurrence。

相对远程搜索：本地 dirty/staged 状态是一等输入，remote 只能加速和共享。

Agent workflow

Agent 任务	推荐命令	输出契约	验证动作
找入口文件	files、find-path、tree	路径、snapshot、ignore 状态、排序规则	必要时 read 目录或文件头
找字符串或报错	find、grep	L0 source_fact，行列范围，index freshness	read file:range
跳转定义	defs	L1P 或 L1S，producer 和 fallback reason	L1S fallback 必须读源码验证
分析影响面	refs、callers、graph traversal	L1P/L0/L2 分开，不混入准确集合	对 L2 逐条 read 验证

算法与架构分析

核心算法路径

Snapshot 构建：解析 repo root，区分 commit、staged、worktree，计算 path、size、mtime、blake3 hash。

Text index：以 n-gram/trigram segment 做 literal、substring、regex prefilter，再读取源码验证 range。

Precise occurrence：优先消费 SCIP/语言服务产物，支撑 defs、refs、symbols。

Parser fallback：tree-sitter 生成声明、symbol、语法范围，但只标注 L1S。

Graph candidate：从 precise occurrence 和 parser facts 派生 property graph，calls/callers/impact 只输出 L2 candidate。

更新生命周期

Manual command：index build/update/status/verify/clean 显式维护本地缓存。

Git hook：pre-commit 读取 staged blob；post-commit promote verified snapshot。

Watcher：只维护 worktree overlay，debounce/coalesce/reconcile 后进入 IndexScheduler。

Atomic publish：temp snapshot + fsync + rename，避免半损坏索引。

Fallback：freshness 失败时读取当前文件实时扫描，不返回旧缓存。

推荐存储分层

```
.code-search/  
  snapshots/<snapshot_id>/  
    manifest.json  
    files.parquet  
    blobs/  
  text/<snapshot_id>/  
    grams.idx  
    docs.idx  
    paths.idx  
  scip/<snapshot_id>/  
    index.scip  
    occurrences.db  
  graph/<snapshot_id>/  
    kuzu/  
      export/nodes.parquet  
      export/edges.parquet  
  working/manifest.json  
  staged/manifest.json
```

架构取舍

层	应该做	不应该做	竞品启发
Text Search	code-specific gram index、regex candidate、source verification	把全文搜索塞进 graph DB	GitHub Blackbird、Zoekt
Symbol/Defs	优先 SCIP/语言服务，fallback tree-sitter	从 AST 猜测 precise refs	Sourcegraph SCIP
Graph	结构遍历、impact、candidate calls、architecture slice	宣称 heuristic 边是完整调用图	CodeGraphContext、GitNexus、CodeGraph
Facts Schema	节点/边带 schema、producer、snapshot、range	随意 JSON、不可追溯派生事实	Glean
Agent API	稳定 JSON、MCP wrapper、LLM 指令、freshness 解释	让 Agent 在多套 shell 输出之间猜	GitNexus、CodeGraph

横向对比矩阵

系统	核心定位	文本搜索	Defs/Refs	Graph	Freshness
GitHub Code Search	平台级远程代码搜索	Rust code-specific n-gram/regex	不是重点	不是重点	大规模索引一致性
Sourcegraph	企业代码搜索 + precise navigation	Zoekt trigram	SCIP/precise navigation	code intel backend	indexed + searcher fallback
CodeGraphContext	可查询代码 property graph	不是重点	tree-sitter + SCIP ingestion	Kuzu/FalkorDB/Neo4j	watcher/ingestion
Glean	typed facts 查询系统	非核心	typed facts	facts query	build/indexer 产物
GitNexus	Agent graph workflow	BM25 + vector hybrid	tree-sitter/provider pipeline	LadybugDB graph	staleness hints + registry
CodeGraph	local-first agent knowledge graph	SQLite FTS5	tree-sitter extraction	SQLite graph tables	OS watcher debounce sync
code-search	Agent 的可验证代码证据层	本地 gram index + source verification	SCIP first, parser fallback	Kuzu-style property graph candidate	commit/staged/worktree + hash/range

定位分析方法论

选择方法：Agent JTBD + 战略画布 + ERRC

这个项目处在多个品类交界处：代码搜索、代码智能、代码图、MCP/Agent 工具。单纯用 SWOT 容易泛化。更合适的方法是先定义 Agent 的 Job-to-be-Done，再用战略画布比较关键竞争因子，最后用 ERRC 明确产品应该消除、降低、提高和创造什么。

JTBD

当 Agent 面对大型代码库时，它需要快速获得可验证代码证据，以便减少盲读、避免幻觉、做出可审计的修改决策。

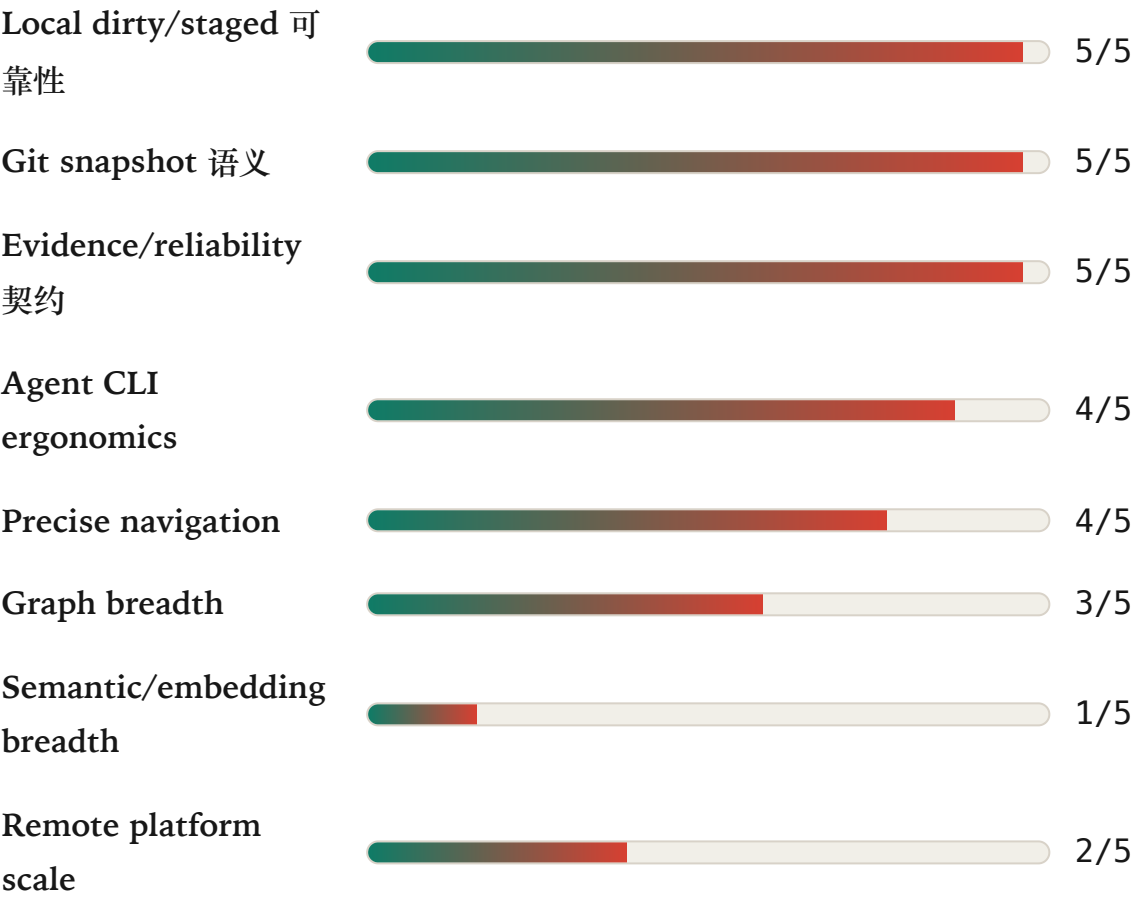
Strategy Canvas

比较维度不按“功能多少”排序，而按 Agent 完成任务时的价值因子排序：本地性、Git 语义、精确性、图广度、证据契约、部署成本。

ERRC

通过 eliminate / reduce / raise / create 形成清晰取舍，避免项目被竞品功能面拖成泛化平台。

战略画布：code-search 应拉高的价值因子



ERRC 取舍

Eliminate 消除

默认 embedding、黑箱 confidence、无来源调用图、remote 覆盖本地状态、提交大型索引 artifact。

Reduce 降低

自动上下文组装、泛化 Web UI、跨仓大平台野心、heuristic 结果在默认输出中的权重。

Raise 提高

snapshot/freshness 验证、统一 JSON schema、staged/working tree 分离、fallback 透明度、Agent 命令覆盖度。

Create 创造

面向 Agent 的 reliability contract、本地可验证代码证据层、watcher + hook 统一 scheduler、候选关系强制验证指令。

竞争优势判断

1. 信任优势

对 Agent 而言，最贵的错误不是“没找到所有候选”，而是把候选当事实并修改代码。`code-search` 的 L0/L1P/L1S/L2 分级、snapshot、range、producer、fallback reason，可以直接降低幻觉和误改概率。

2. 本地状态优势

GitHub/Sourcegraph 这类远程系统天然更擅长默认分支和托管代码。Agent 实际修改代码时，关键状态是 dirty worktree、staged snapshot 和当前 HEAD 的组合。`code-search` 把这些作为一等事实源。

3. 工具面优势

Agent 不应该在 rg、fd、ls、cat、IDE 和 MCP 间切换输出格式。一个统一 CLI/JSON 命令面能减少 prompt glue 和解析失败。

4. 架构弹性优势

项目采用 source snapshot、text index、SCIP occurrence、parser facts、property graph 分层。这让每层可以替换实现，而不把搜索、跳转、图遍历和 freshness 混在一个数据库里。

定位护城河

护城河	为什么竞品不容易直接覆盖	本项目必须兑现的工程要求
Git-first freshness contract	远程平台和 graph 产品通常不以 staged/dirty/worktree 细粒度为首要模型。	staged 通过 git blob 读取；每条结果携带 snapshot/file hash/range。
Reliability-labeled output	很多工具输出“相关结果”，但不会为 LLM 明确区分事实、parser fact 和候选推断。	所有命令必须有 reliability、exact、fallback_reason 和 LLM 指令。
Unified Agent command surface	传统工具分散，Web 产品偏人类 UI，graph 工具偏结构查询。	兼容 grep/find-path/list/read，但输出统一 schema。
Deterministic-first extensibility	embedding/hybrid 产品容易在相似度与真实性之间模糊边界。	embedding 只能 opt-in；默认路径必须可证明、可回退、可审计。

风险与路线建议

短期风险

当前仓库仍是 scaffold。如果没有先做 L0 search/read/path 的可用闭环，架构叙事会先于产品体验。

中期风险

precise navigation 依赖语言生态和 SCIP indexer。需要清晰 fallback，不要让 tree-sitter 输出伪装成精确引用。

长期风险

graph breadth 会诱惑项目追 GitNexus/CodeGraph 的复杂功能面。必须坚持 graph candidate 和 provenance 边界。

建议路线

阶段	目标	关键验收	竞争意义
Phase 1	实现 L0 搜索、路径、读取、changed/status	无索引也能跑；JSON schema 稳定；行列范围准确；快照测试覆盖。	形成 Agent 统一命令面的最低可用价值。
Phase 2	实现 IndexScheduler、hook、watcher、freshness	staged/worktree 不混用；过期自动 fallback；status 能解释原因。	建立远程工具很难复制的本地 Git 语义优势。
Phase 3	接入 tree-sitter symbols 和 SCIP precise occurrence	L1P/L1S 分开；parser warning 和 unsupported language 可见。	进入 IDE 级跳转体验，但保持准确性诚实。
Phase 4	增加 graph candidate、impact traversal、MCP wrapper	L2 永不 exact；每条边有 provenance；MCP 与 CLI schema 一致。	进入 Agent graph 竞争区，但以证据契约区别于泛化图工具。

资料来源

本报告基于当前仓库文档整理，没有重新联网核验各竞品的最新实现细节；竞品描述以项目文档中的公开资料摘要为准。

- [docs/00-design-summary.md](#)
- [docs/01-product-positioning.md](#)
- [docs/02-reliability-model.md](#)
- [docs/03-command-design.md](#)
- [docs/04-rust-architecture.md](#)
- [docs/05-implementation-roadmap.md](#)
- [docs/06-index-and-hooks.md](#)
- [docs/07-index-algorithm-analysis.md](#)
- [docs/08-reference-architecture.md](#)
- [docs/09-agent-ide-principles.md](#)
- [docs/10-agent-search-command-surface.md](#)
- [docs/11-watcher-design.md](#)